



GOVERNMENT POLYTECHNIC MUZAFFARPUR

(Under Department of Science, Technology & Technical
Education Bihar)



LAB MANUAL

Introduction to Artificial Intelligence Lab

Course Code: P2600100

INDEX

Sr. No.	Title of Practical	CO
1	Navigate local GUI directories, perform folder operations and extract hardware configuration	CO1
2	Perform local system administration and organize directory structures for data workflows	CO1
3	Configure a collaborative cloud directory and implement shared access permissions	CO1
4	Develop a simple image classification model using Google Teachable Machine	CO2
5	Import and visualize engineering measurement data using Orange Data Mining Toolkit	CO2
6	Perform image annotation and generate object bounding boxes using a zero-code AI tool	CO3
7	Apply prompt engineering using Generative AI for engineering design visualizations	CO4
8	Use Python variables, data types, arithmetic operators and type casting for engineering calculations	CO5
9	Create interactive Python programs using input(), explicit type casting and f-strings	CO5
10	Implement decision-making programs using if-else and nested if statements	CO5
11	Use for and while loops to automate repetitive computations and generate sequences	CO5
12	Develop a menu-driven Python program using loops and conditional statements	CO5
13	Use Python lists to store, process and analyze numerical data	CO5

COURSE INFORMATION

Section A: Course Details

Particular	Details
Course Code	P2600100
Course Title	Introduction to Artificial Intelligence
Programme	Diploma in Computer Science & Engineering
Semester	I (First)
Lab Instruction	04 Hours/Week
Progressive Lab Assessment (PLA)	20 Marks
Pre-requisite	Basic digital literacy, familiarity with computer operations, and awareness of internet usage.

Course Outcomes (COs)

CO No.	Course Outcome Statement
CO1	Explain the fundamentals of information technology, computer systems, operating systems, cloud computing, and cybersecurity practices.
CO2	Explain the fundamental concepts of Artificial Intelligence, data handling, AI learning processes, and AI applications.
CO3	Describe the basic principles and applications of Computer Vision, Optical Character Recognition (OCR), and Natural Language Processing (NLP).
CO4	Apply conversational AI and generative AI tools using prompt engineering techniques while recognizing their capabilities, limitations, and responsible use.
CO5	Develop simple Python programs using variables, operators, decision-making constructs, loops, and lists for problem solving and data processing.

Programme Outcomes (POs)

PO	Programme Outcome
PO1	Basic Science and Engineering Knowledge
PO2	Problem Analysis
PO3	Design/Development of Solutions
PO4	Investigation of Problem with code, test & measurement
PO5	Engineering Tools Usage
PO6	Sustainability Awareness
PO7	Professional Ethics
PO8	Individual and Collaborative Team Work
PO9	Communication Skills
PO10	Project Management
PO11	Life-Long Learning

Section B: Course Articulation Matrix (CO-PO Matrix)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	1	-	-	2	-	1	1	1	-	1
CO2	3	2	1	1	2	-	-	1	1	-	1
CO3	3	2	1	1	2	-	1	-	1	-	1
CO4	2	2	2	1	2	1	-	2	1	1	2
CO5	2	2	3	2	3	-	1	1	1	1	2

Legend: 3 = High Correlation | 2 = Medium Correlation | 1 = Low Correlation | - = No Mapping

Section C: General Lab Instructions

- Read the practical thoroughly before coming to the laboratory.
- Use institutional/login credentials responsibly and never share passwords.
- Keep local and cloud files organized using meaningful names and folder structures.
- For AI tools, use only appropriate, non-sensitive data and verify generated outputs before accepting them.
- Do not upload personal, confidential, copyrighted, or restricted data to public AI services.
- For zero-code AI activities, record dataset details, settings, observations, and limitations.
- For Python practicals, write the aim, code, output/observations, and conclusion in the lab journal.
- Viva-voce questions will be asked at the end of each practical session.
- Maintain the lab journal up to date and submit it on time.

Assessment

S.No.	Category	Criteria of Assessment	% Weightage
1	AI / Digital Tool Execution	Correct use of prescribed tool, workflow or implementation	20
		Independent execution, configuration and troubleshooting	15
		Testing with suitable inputs / data	10
2	Result and Documentation	Accuracy and relevance of output / results	15
		Analysis, interpretation and correctness of approach	15
		Completeness of lab record and evidence	10
3	Viva-Voce	Conceptual understanding and responsible AI awareness	15

Rubrics for Evaluation (Progressive Lab Assessment - 20 Marks)

S.No.	Name of Student	AI / Tool Execution (8)	Result & Documentation (8)	Viva (4)	Total (20)

Practical No: 1

1. Aim / Practical Statement

Navigate local GUI directories, perform folder operations and extract hardware configuration

2. Practical Significance

File and hardware awareness is essential for all later AI and programming work. This develops confidence in navigating the operating system, organizing local data, and identifying the CPU, RAM and storage available for computational tasks.

3. Relevant Theory

GUI File System

- A graphical file manager represents drives, folders and files using icons and directory paths.
- Common operations include create, copy, move, rename, compress (ZIP) and extract.

Hardware Configuration

- CPU performs instruction processing; RAM provides temporary working memory; storage retains files persistently.
- System information utilities report processor, installed memory, operating system and storage capacity.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	1	-	-	2	-	1	1	1	-	1

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Navigate local directories using a GUI file manager.
- Create, copy, move, rename, compress and extract folders/files.
- Identify CPU, RAM and storage configuration of the computer.

6. Resources Required

Sr.	Resource	Specification
1	Computer System	Intel Core i5/i7 or equivalent, 8 GB RAM or above
2	Operating System	Windows 10/11 or Linux
3	Internet	Broadband / Wi-Fi / LAN connectivity

7. Suggested Procedure

1. Open File Explorer/File Manager and identify the user home directory.
2. Create a folder named AI_Lab and subfolders Data, Images, Python and Reports.
3. Create a sample text file; copy, move and rename it.
4. Compress one folder to ZIP and extract it to another location.
5. Open system information/settings and record CPU, RAM and storage details.
6. Capture/record observations in the lab journal.

8. Activity / Implementation

- Suggested structure: AI_Lab/Data, AI_Lab/Images, AI_Lab/Python, AI_Lab/Reports.
- Record hardware configuration in a small table.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Create folders	AI_Lab folder hierarchy created successfully
2	Copy/Move/Rename	File operations completed without data loss
3	ZIP/Extract	Archive created and extracted successfully
4	System information	CPU, RAM and storage parameters recorded

10. Result

Local GUI directory operations and hardware configuration identification were performed successfully.

11. Interpretation of Results

- Organized directories reduce errors in later data and AI workflows.
- Hardware specifications help determine whether a system can run computationally demanding applications.

12. Cautions

- Do not delete or move system folders.
- Verify the destination before moving important files.
- Do not expose device serial numbers or other sensitive identifiers in public reports.

13. Viva-Voce Questions

S.No .	Viva-Voce Question	R	U	A
1	State the full form of RAM.	✓		
2	Name two common file operations.	✓		
3	Name the three main hardware components recorded in this practical.	✓		
4	Differentiate between a file and a folder.		✓	
5	Explain the purpose of ZIP compression.		✓	
6	Differentiate RAM and storage.		✓	
7	Create a suitable folder structure for an AI lab project.			✓
8	Select an appropriate file operation to relocate a file without keeping the original copy.			✓
9	Use system information to identify the RAM and storage capacity of a computer.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 2

1. Aim / Practical Statement

Perform local system administration and organize directory structures for data workflows

2. Practical Significance

Well-structured directories and basic local administration improve reliability, traceability and efficiency when working with datasets, reports and program files.

3. Relevant Theory

File Paths

- An absolute path identifies a location from the root/drive; a relative path is interpreted from the current working location.

Local Administration

- Shortcuts, naming conventions, permissions and storage cleanup support efficient local workflows.
- A consistent project structure separates raw data, processed data, source code and outputs.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	1	-	-	2	-	1	1	1	-	1

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Organize directories for an engineering/AI data workflow.
- Differentiate absolute and relative paths.
- Create shortcuts and inspect file properties/permissions.

6. Resources Required

Sr .	Resource	Specification
1	Computer System	Intel Core i5/i7 or equivalent, 8 GB RAM or above
2	Operating System	Windows 10/11 or Linux
3	Internet	Broadband / Wi-Fi / LAN connectivity

7. Suggested Procedure

1. Create a project folder named AI_Project.
2. Inside it create Raw_Data, Processed_Data, Models, Outputs and Documentation.
3. Place sample files in appropriate folders and rename them consistently.
4. Create a desktop shortcut to AI_Project.
5. Inspect file/folder properties and available access attributes.
6. Write one absolute path and one relative path for a sample file.

8. Activity / Implementation

- Use a naming convention such as dataset_v1.csv, report_v1.docx and output_01.png.
- Draw the final directory tree in the lab record.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Project structure	Five functional folders created
2	Naming	Files renamed consistently

Sr. No.	Step / Operation Performed	Output / Behaviour
3	Shortcut	Project shortcut opens correct directory
4	Paths	Absolute and relative paths identified correctly

10. Result

A structured local directory was created and basic administration tasks were completed successfully.

11. Interpretation of Results

- Separation of raw data and outputs prevents accidental overwriting.
- Version-oriented naming improves traceability and reproducibility.

12. Cautions

- Avoid changing permissions on system directories.
- Keep an untouched copy of raw data.
- Use meaningful names and avoid ambiguous files such as final_final2.

13. Viva-Voce Questions

S.No .	Viva-Voce Question	R	U	A
1	Define an absolute path.	✓		
2	Define a relative path.	✓		
3	Name two folders commonly used in a data workflow.	✓		
4	Explain the purpose of a shortcut.		✓	
5	Discuss the importance of naming conventions.		✓	
6	Discuss the importance of keeping raw data unchanged.		✓	
7	Design a directory structure for an image-classification project.			✓
8	Write one suitable filename using a version-oriented naming convention.			✓
9	Classify a given file path as absolute or relative.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 3

1. Aim / Practical Statement

Configure a collaborative cloud directory and implement shared access permissions

2. Practical Significance

Cloud collaboration allows students and engineering teams to store, share and edit documents from multiple locations while controlling who can view or modify information.

3. Relevant Theory

Cloud Storage

- Cloud services such as Google Drive and Microsoft OneDrive store files on remote infrastructure accessible through the internet.

Collaboration Permissions

- Viewers can read, Commenters can review/comment, and Editor can modify content.
- Shared links must be configured carefully to prevent unintended access.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	1	-	-	2	-	1	1	1	-	1

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Create a collaborative cloud directory.
- Upload multiple file formats and share them with appropriate permissions.
- Differentiate Viewer, Commenter and Editor access.

6. Resources Required

Sr.	Resource	Specification
1	Computer / Mobile Device	Browser-enabled system
2	Cloud Platform	Google Drive / Microsoft OneDrive
3	Browser	Chrome / Edge / Firefox
4	Internet	Reliable internet connection

7. Suggested Procedure

1. Create a folder named AI_Lab_Cloud in the approved cloud platform.
2. Upload a document, spreadsheet/image and PDF/sample file.
3. Create a subfolder for Team_Work.
4. Share a file/folder first as Viewer, then Commenter, then Editor with a lab partner.
5. Observe which actions are permitted at each access level.
6. Revoke or restrict access after the activity.

8. Activity / Implementation

- Prepare a permission comparison table for Viewer, Commenter and Editor.
- Record the file types uploaded and the sharing method used.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Multi-format upload	Files uploaded successfully
2	Viewer access	Content viewable but not editable
3	Commenter access	Comments allowed without direct editing

Sr. No.	Step / Operation Performed	Output / Behaviour
4	Editor access	Content editable
5	Access control	Sharing restricted/revoked after test

10. Result

A collaborative cloud directory was configured and different sharing permissions were tested successfully.

11. Interpretation of Results

- Access rights should follow the principle of least privilege.
- Cloud collaboration supports real-time teamwork but requires careful privacy and link-sharing practices.

12. Cautions

- Never share passwords or OTPs.
- Do not upload confidential/personal records for practice.
- Avoid public “anyone with the link can edit” settings unless specifically required.

13. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	Define cloud storage.	✓		
2	Name two cloud storage platforms.	✓		
3	State the three common cloud access levels.	✓		
4	Differentiate Viewer and Editor permissions.		✓	
5	Explain the permissions available to a Commenter.		✓	
6	Discuss the risks associated with public edit links.		✓	
7	Select a suitable permission for a user who only needs to read a shared file.			✓
8	Configure a shared folder so a teammate can edit its contents.			✓
9	Revoke shared access after completing a collaborative activity.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 4

1. Aim / Practical Statement

Develop a simple image classification model using Google Teachable Machine

2. Practical Significance

Image classification is a beginner-friendly way to observe how a machine learning system learns visual patterns from labelled examples without writing model code.

3. Relevant Theory

Image Classification

- Image classification assigns an input image to one of the learned classes.

Training and Testing

- Training examples are used to learn patterns; testing examples are used to check performance on unseen samples.
- Balanced, varied and correctly labelled examples generally improve model behaviour.

Zero-Code ML

- Google Teachable Machine provides a web interface for collecting examples, training a model and testing predictions.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO2	3	2	1	1	2	-	-	1	1	-	1

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Create labelled image classes in a zero-code ML tool.
- Train a simple image classification model.
- Test the model with unseen images and interpret confidence scores.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Webcam-enabled computer preferred
2	Platform	Google Teachable Machine
3	Dataset	Two or more non-sensitive image classes
4	Internet	Broadband / Wi-Fi

7. Suggested Procedure

1. Open Google Teachable Machine and select an Image Project.
2. Create at least two classes (for example, pen and notebook).
3. Collect/upload multiple varied images for each class.
4. Train the model using the default training workflow.
5. Test with new examples not used during training.
6. Record predicted class and confidence for at least five tests.
7. Discuss one case where the model is uncertain or incorrect.

8. Activity / Implementation

- Do not use faces or personal/sensitive images unless institutionally approved.
- Use clearly distinct object classes and vary angle/background where possible.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Training	Model trained on labelled examples
2	Known-style sample	Correct class with high confidence expected
3	Unseen sample	Prediction recorded
4	Ambiguous sample	Lower/competing confidence observed
5	Error analysis	Possible reason for wrong prediction identified

10. Result

A simple zero-code image classification model was trained and evaluated using new image samples.

11. Interpretation of Results

- The model learns statistical visual patterns rather than human-like meaning.
- Training data quality, variety and class balance influence predictions.
- Confidence is not a guarantee of correctness.

12. Cautions

- Avoid sensitive or copyrighted datasets.
- Keep classes reasonably balanced.
- Always test with unseen samples instead of judging only training examples.

13. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define image classification.	✓		
2	Define training data.	✓		
3	Name the zero-code tool used for image classification in this practical.	✓		
4	Explain the meaning of a confidence score.		✓	
5	Discuss the role of testing data in model evaluation.		✓	
6	Explain overfitting in simple terms.		✓	
7	Select suitable image classes for a simple Teachable Machine project.			✓
8	Test a trained model using an unseen image and record the predicted class.			✓
9	Identify a likely reason for an incorrect prediction from an imbalanced dataset.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 5

1. Aim / Practical Statement

Import and visualize engineering measurement data using Orange Data Mining Toolkit

2. Practical Significance

Engineering decisions often begin with understanding data. Visualizing a CSV dataset helps identify trends, clusters, relationships and unusual values before building AI models.

3. Relevant Theory

Structured Data

- CSV data is commonly organized in rows (records) and columns (features/variables).

Visualization

- Scatter plots show relationships between two numerical variables; charts help reveal distribution and trends.

Orange Toolkit

- Orange is a visual, widget-based data mining environment that supports importing, exploring and modelling data without extensive coding.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO2	3	2	1	1	2	-	-	1	1	-	1

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Import a CSV dataset into Orange.
- Visualize variables using suitable widgets.
- Interpret trends and relationships among measurements.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Intel Core i5 or equivalent
2	Software	Orange Data Mining Toolkit
3	Dataset	Simple engineering measurement CSV
4	Optional	Spreadsheet software for viewing CSV

7. Suggested Procedure

1. Prepare/open a CSV with at least two numerical variables and one label/category if available.
2. Open Orange and place the File widget on the canvas.
3. Load the CSV and inspect detected features.
4. Connect Data Table and review rows/columns.
5. Connect Scatter Plot and select two numerical variables.
6. Optionally use Distributions/Box Plot for another view.
7. Record at least three observations from the visualizations.

8. Activity / Implementation

- Example data may contain temperature, voltage, current, pressure or other non-sensitive measurements.

- Save a screenshot of the Orange workflow if permitted.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	CSV import	Dataset loaded and variables detected
2	Data Table	Rows and values inspected
3	Scatter Plot	Relationship between two variables visualized
4	Trend analysis	Increasing/decreasing/no clear relationship identified

10. Result

Engineering measurement data was imported, visualized and interpreted using Orange Data Mining Toolkit.

11. Interpretation of Results

- Visualization can reveal patterns before formal modelling.
- Correlation in a plot does not automatically prove causation.
- Missing values and outliers should be checked before drawing conclusions.

12. Cautions

- Verify units and column meanings.
- Do not alter raw data without recording the change.
- Avoid claiming causal relationships from a simple plot alone.

13. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	State the full form of CSV.	✓		
2	Define structured data.	✓		
3	Name two visualization widgets used in Orange.	✓		
4	Explain the information represented by a scatter plot.		✓	
5	Explain the meaning of a feature in a dataset.		✓	
6	Differentiate correlation and causation.		✓	
7	Select a suitable Orange widget to visualize two numerical variables.			✓
8	Identify an outlier from a given set of measurement values.			✓
9	Interpret a scatter plot as showing an increasing, decreasing, or unclear relationship.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 6

1. Aim / Practical Statement

Perform image annotation and generate object bounding boxes using a zero-code AI tool

2. Practical Significance

Object detection systems require labelled examples that specify both the object category and its location. Bounding-box annotation introduces the data preparation process used in many computer vision applications.

3. Relevant Theory

Object Detection

- Object detection identifies both what an object is and where it appears in an image.

Bounding Box

- A bounding box is a rectangular region drawn around an object and associated with a class label.

Annotation Quality

- Consistent labels and tight boxes improve dataset quality; missed objects or incorrect boxes introduce annotation errors.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO3	3	2	1	1	2	-	1	-	1	-	1

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Annotate objects in engineering images using bounding boxes.
- Assign consistent class labels.
- Identify common annotation errors.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Browser-enabled system
2	Annotation Tool	Roboflow or equivalent zero-code annotation tool
3	Images	Non-sensitive engineering/object images
4	Internet	Required for web-based tool

7. Suggested Procedure

1. Create/open an annotation project in the approved tool.
2. Upload a small set of non-sensitive images.
3. Define one or more object classes.
4. Draw a tight bounding box around each target object and assign the correct label.
5. Repeat consistently across images.
6. Review annotations for missed objects, loose boxes and wrong labels.
7. Export/record the annotation summary if permitted.

8. Activity / Implementation

- Suggested objects: tools, components, stationery or lab objects.
- Record examples of correct and incorrect annotation practices.

9. Sample Output / Observations

Sr. No .	Step / Operation Performed	Output / Behaviour
1	Class definition	Consistent labels created
2	Bounding boxes	Target objects enclosed appropriately
3	Review	Missed/wrong/loose annotations checked
4	Dataset readiness	Annotated examples ready for later model training

10. Result

Object images were annotated with labelled bounding boxes and common annotation errors were analyzed.

11. Interpretation of Results

- Object detection differs from classification because it includes localization.
- Annotation consistency is a key part of computer vision dataset quality.

12. Cautions

- Do not upload confidential engineering drawings or personal images.
- Boxes should include the complete target object without excessive background.
- Use the same label spelling consistently.

13. Viva-Voce Questions

S. No .	Viva-Voce Question	R	U	A
1	Define object detection.	✓		
2	Define a bounding box.	✓		
3	Name the rectangular annotation used to locate an object.	✓		
4	Differentiate image classification and object detection.		✓	
5	Explain the role of class labels in object detection.		✓	
6	Discuss the importance of tight bounding boxes.		✓	
7	Draw an appropriate bounding box around a target object in an image.			✓
8	Identify an annotation error in a sample labelled image.			✓
9	Assign a consistent class label to repeated instances of the same object.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 7

1. Aim / Practical Statement

Apply prompt engineering using Generative AI for engineering design visualizations

2. Practical Significance

Prompt engineering helps users communicate requirements to generative AI systems. Structured prompts are especially useful for generating and comparing engineering design concepts while retaining human review and responsibility.

3. Relevant Theory

Prompt Engineering

- An effective prompt clearly states the task, context, constraints and desired output format.

Zero-Shot Prompting

- Zero-shot prompting asks a model to perform a task directly without providing worked examples.

Generative AI Limitations

- Outputs may contain hallucinations, design errors, bias or unsupported claims and therefore require verification.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO4	2	2	2	1	2	1	-	2	1	1	2

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Construct clear structured prompts.
- Generate and compare engineering design visualizations/alternatives using a generative AI tool.
- Evaluate outputs for relevance, limitations and responsible use.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Internet-enabled computer
2	Generative AI	Institutionally permitted generative AI tool
3	Prompt Record	Lab journal / document for prompt-output comparison

7. Suggested Procedure

1. Select a simple non-safety-critical design visualization task (for example, a labelled smart classroom layout concept).
2. Write Prompt A with only a short task statement.
3. Write Prompt B with role/context, specific requirements, constraints and output format.
4. Generate outputs for both prompts using the approved tool.
5. Compare completeness, relevance and adherence to constraints.
6. Identify any hallucinated, impractical or unverifiable elements.
7. Revise the structured prompt once and record the improvement.

8. Activity / Implementation

- Prompt template: Task + Context + Specific Instructions + Constraints + Desired Output.
- All AI-generated engineering content must be treated as a draft concept requiring verification.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Prompt A	General output generated
2	Prompt B	More constrained/structured output generated
3	Comparison	Differences in relevance and constraint-following recorded
4	Verification	Potential errors/unsupported details identified
5	Revision	Improved prompt tested

10. Result

Structured prompt engineering was applied to generate and evaluate engineering design alternatives using a generative AI tool.

11. Interpretation of Results

- More explicit context and constraints often improve task alignment.
- Generative AI can support ideation but does not replace engineering validation.
- Responsible use requires source checking, privacy awareness and disclosure where appropriate.

12. Cautions

- Never enter confidential or personal data into public AI tools.
- Do not rely on generated technical specifications without verification.
- Respect copyright and institutional policies when using generated content.

13. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define prompt engineering.	✓		
2	Define zero-shot prompting.	✓		
3	Name the main components of an effective prompt.	✓		
4	Explain the meaning of AI hallucination.		✓	
5	Discuss the importance of fact verification in Generative AI.		✓	
6	Explain the role of constraints in improving a prompt.		✓	
7	Construct a structured prompt for a labelled smart-classroom layout.			✓
8	Revise a vague prompt by adding context, constraints, and desired output.			✓
9	Identify an unverifiable or hallucinated element in an AI-generated design response.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 8

1. Aim / Practical Statement

Use Python variables, data types, arithmetic operators and type casting for engineering calculations

2. Practical Significance

Variables, data types and operators are the foundation of Python-based computational thinking. Engineering formulas can be translated into clear executable expressions.

3. Relevant Theory

Variables and Data Types

- Python variables store values; common primary types include int, float, str and bool.

Operators

- Arithmetic operators perform calculations; relational and logical operators support comparisons and conditions.

Type Casting

- Explicit conversion such as int(), float() and str() changes the representation/type of a value when appropriate.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO5	2	2	3	2	3	-	1	1	1	1	2

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Declare and use Python variables of primary data types.
- Apply arithmetic operators to engineering calculations.
- Use explicit type casting and evaluate simple formulas.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Intel Core i3/i5 or above
2	Python Environment	Python 3.x / IDLE / Jupyter Notebook / VS Code / PyCharm

7. Suggested Procedure

1. Open the Python environment and create a new script/notebook.
2. Declare integer, float, string and boolean variables and display their types.
3. Calculate area and perimeter of a rectangle using stored values.
4. Calculate average of three measurements.
5. Convert a numeric string to float and perform a unit conversion.
6. Run the program and verify outputs manually.

8. Activity / Implementation

- Example formulas: area = length * width; perimeter = 2*(length+width); average = (a+b+c)/3.
- Use type() to observe data types.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Data types	int, float, str and bool values created
2	Area/Perimeter	Calculated values match manual calculation
3	Average	Mean value calculated
4	Type casting	String numeric value converted successfully

10. Result

Python variables, data types, arithmetic operations and explicit type casting were used successfully for simple engineering calculations.

11. Interpretation of Results

- Python determines variable types dynamically from assigned values.
- Explicit casting is useful when converting user/text data into numerical form.

12. Cautions

- Use meaningful variable names.
- Check units before applying formulas.
- Avoid casting invalid text directly to numbers without validation.

13. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Name four primary Python data types.	✓		
2	State the function used to identify the data type of a value.	✓		
3	Name the symbols used for addition and multiplication in Python.	✓		
4	Differentiate int and float.		✓	
5	Explain the purpose of explicit type casting.		✓	
6	Explain the importance of units in engineering calculations.		✓	
7	Write a Python expression to calculate the area of a rectangle.			✓
8	Convert the string '12.5' to a floating-point value in Python.			✓
9	Predict the data type of the result of 10/2 in Python 3.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 9

1. Aim / Practical Statement

Create interactive Python programs using input(), explicit type casting and f-strings

2. Practical Significance

Interactive programs accept user data, convert it to suitable types and present readable results. These skills are essential for small engineering utilities and later data-processing programs.

3. Relevant Theory

Input

- input() captures console input as a string. Numerical calculations generally require explicit conversion.

Formatted Output

- f-strings embed variables and expressions inside readable output text using braces.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO5	2	2	3	2	3	-	1	1	1	1	2

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Accept user input using input().
- Convert input using explicit type casting.
- Display clean formatted output using f-strings.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Standard laboratory computer
2	Python Environment	Python 3.x / IDLE / VS Code / Jupyter

7. Suggested Procedure

1. Write a program to accept a student/user name and two numerical measurements.
2. Convert measurement strings to float.
3. Compute sum and average.
4. Display the values and result using f-strings with suitable labels/units.
5. Test with at least three valid input sets.

8. Activity / Implementation

- Example: temperature readings, voltage readings or generic measurements.
- Use formatting such as {average:.2f} for two decimal places.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Text input	Name captured as string
2	Numeric input	Values converted to float
3	Computation	Sum/average computed
4	Formatted output	Readable labelled result displayed

10. Result

An interactive Python program was created using input(), explicit type casting and formatted string literals.

11. Interpretation of Results

- input() returns text, so conversion is required for arithmetic.
- Formatted output improves readability and reduces ambiguity.

12. Cautions

- Enter values in the expected numeric format.
- Handle units consistently.
- Do not collect unnecessary personal information in practice programs.

13. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	State the data type returned by input().	✓		
2	State the function used to accept keyboard input in Python.	✓		
3	Name the prefix used before a formatted string literal.	✓		
4	Explain the purpose of float(input()) for numerical input.		✓	
5	Explain the purpose of an f-string.		✓	
6	Discuss the importance of including units in program output.		✓	
7	Write an f-string to display a value up to two decimal places.			✓
8	Create a statement that accepts age as an integer from the keyboard.			✓
9	Identify the likely error when alphabetic text is converted using float().			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 10

1. Aim / Practical Statement

Implement decision-making programs using if-else and nested if statements

2. Practical Significance

Decision-making allows programs to select actions based on conditions. In engineering automation, conditional logic can represent threshold checks and simple safety alarms.

3. Relevant Theory

Conditional Statements

- if executes a block when a condition is true; elif checks additional conditions; else handles remaining cases.

Nested Conditions

- An if statement can appear inside another to represent multi-level decisions.

Threshold Logic

- Relational operators compare measurements with limits to trigger messages or actions.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO5	2	2	3	2	3	-	1	1	1	1	2

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Use if-else and nested if statements.
- Implement a simple threshold-based decision.
- Test boundary and normal cases.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Standard laboratory computer
2	Python Environment	Python 3.x / IDLE / VS Code

7. Suggested Procedure

1. Write a program to accept a measurement such as temperature.
2. Use if-elif-else to classify it as normal, warning or critical using teacher-defined example thresholds.
3. Add a nested condition for an additional status check.
4. Test values below, at and above each threshold.
5. Record the decision path for each test.

8. Activity / Implementation

- Threshold values used in this practical are educational examples only and must not be treated as real safety limits.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Normal input	Normal status displayed
2	Boundary input	Expected branch selected
3	High input	Warning/critical message displayed

Sr. No.	Step / Operation Performed	Output / Behaviour
4	Nested condition	Secondary condition evaluated

10. Result

Decision-making logic using if-else and nested conditions was implemented and tested with multiple inputs.

11. Interpretation of Results

- Boundary testing is important because comparison operators can change behaviour exactly at threshold values.
- Real safety systems require validated thresholds, sensors and fail-safe design beyond this classroom example.

12. Cautions

- Use only teacher-defined illustrative thresholds.
- Check whether comparisons use <, <=, > or >= as intended.
- Do not present this exercise as a certified safety controller.

13. Viva-Voce Questions

S.No .	Viva-Voce Question	R	U	A
1	State the keyword used for conditional execution in Python.	✓		
2	Name the keyword used for an alternative condition.	✓		
3	Define a nested if statement.	✓		
4	Explain the purpose of the if statement.		✓	
5	Differentiate elif and else.		✓	
6	Explain the meaning of a boundary value in decision-making programs.		✓	
7	Write a condition for x between 10 and 20 inclusive.			✓
8	Construct an if-else condition to classify a number as positive or non-positive.			✓
9	Test a threshold condition using a value exactly equal to the specified limit.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 11

1. Aim / Practical Statement

Use for and while loops to automate repetitive computations and generate sequences

2. Practical Significance

Loops automate repeated calculations and are useful for numerical sequences, repeated conversions and processing batches of measurements.

3. Relevant Theory

for Loop

- A for loop iterates over a sequence or range of values and is suitable when the number of repetitions is known.

while Loop

- A while loop repeats while a condition remains true and is useful when repetitions depend on a condition.

range()

- range(start, stop, step) generates integer sequences commonly used with for loops.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO5	2	2	3	2	3	-	1	1	1	1	2

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Use for and while loops for repetitive computations.
- Generate numerical sequences using range().
- Compare definite and condition-controlled repetition.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Standard laboratory computer
2	Python Environment	Python 3.x / IDLE / VS Code

7. Suggested Procedure

1. Use a for loop to print a sequence from 1 to 10.
2. Use range() to generate even numbers within a selected interval.
3. Use a for loop to convert a list/range of values using a simple formula.
4. Use a while loop to count down from a selected value.
5. Test loop start, stop and step behaviour.

8. Activity / Implementation

- Example conversion: Celsius to Fahrenheit using $F = (C \times 9/5) + 32$ for a sequence of illustrative values.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	for loop	Sequence generated correctly
2	range step	Even-number sequence generated
3	Conversion loop	Repeated calculations completed
4	while loop	Countdown terminates correctly

10. Result

for and while loops were used to automate repetitive calculations and generate numerical sequences.

11. Interpretation of Results

- for loops are convenient for definite iteration; while loops depend on a condition.
- Incorrect while-loop updates can cause infinite loops.

12. Cautions

- Ensure while-loop variables are updated.
- Check range() stop behaviour (stop value is excluded).
- Avoid unnecessarily large loops during classroom exercises.

13. Viva-Voce Questions

S. No.	Viva-Voce Question	R	U	A
1	Name the two basic loop statements in Python.	✓		
2	State the function commonly used to generate a numerical sequence.	✓		
3	State whether the stop value is included in range().	✓		
4	Differentiate for and while loops.		✓	
5	Explain a common cause of an infinite loop.		✓	
6	Explain the use of loops for automating repetitive computations.		✓	
7	Write a loop to print five multiples of 3.			✓
8	Determine the sequence generated by range(1,10,2).			✓
9	Use a while loop to print the numbers 1 to 5.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 12

1. Aim / Practical Statement

Develop a menu-driven Python program using loops and conditional statements

2. Practical Significance

Menu-driven programs combine input, conditions and loops into a small interactive application, strengthening structured problem solving and program control flow.

3. Relevant Theory

Menu-Driven Program

- A menu presents numbered choices and executes the selected operation.

Control Flow

- A loop keeps the menu active; if-elif-else selects the requested operation; a specific option exits the program.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO5	2	2	3	2	3	-	1	1	1	1	2

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Design a menu-driven Python program.
- Combine loops with conditional statements.
- Validate simple user choices.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Standard laboratory computer
2	Python Environment	Python 3.x / IDLE / VS Code

7. Suggested Procedure

1. Display a menu: 1-Add, 2-Subtract, 3-Multiply, 4-Divide, 5-Exit.
2. Use a while loop to keep the menu active until Exit is selected.
3. Accept two numerical inputs for arithmetic operations.
4. Use if-elif-else to execute the selected operation.
5. Check for division by zero.
6. Display an invalid-choice message for unsupported menu numbers.
7. Test all menu options.

8. Activity / Implementation

- Keep each operation simple and clearly labelled.
- Optional extension: place operations in separate functions after the basic version works.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	Add/Subtract	Correct arithmetic results
2	Multiply	Correct product
3	Divide	Correct quotient / zero check

Sr. No.	Step / Operation Performed	Output / Behaviour
4	Invalid choice	User-friendly message displayed
5	Exit	Loop terminates normally

10. Result

A menu-driven Python application was developed using loops and conditional statements and tested for valid and invalid choices.

11. Interpretation of Results

- Combining repetition and selection produces interactive applications.
- Input validation prevents common runtime errors such as division by zero.

12. Cautions

- Always include an exit condition.
- Validate division denominator before division.
- Keep menu numbering and branch logic consistent.

13. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define a menu-driven program.	✓		
2	Define input validation.	✓		
3	Name two arithmetic operations commonly included in a calculator menu.	✓		
4	Explain the purpose of using a loop around a menu.		✓	
5	Explain the need for an exit option in a menu-driven program.		✓	
6	Explain the need for division-by-zero handling.		✓	
7	Design a menu containing Add, Subtract, Multiply, Divide, and Exit options.			✓
8	Write a condition that prevents division when the divisor is zero.			✓
9	Select an appropriate response for an invalid menu choice entered by a user.			✓

Legend : R = Remember | U = Understand | A = Apply

Practical No: 13

1. Aim / Practical Statement

Use Python lists to store, process and analyze numerical data

2. Practical Significance

Lists provide sequential storage for multiple measurements and support basic analysis such as minimum, maximum, sum and average. They are a foundation for later data processing.

3. Relevant Theory

Python Lists

- A list is an ordered, mutable collection written using square brackets.

List Operations

- `append()` adds an item; `len()` returns item count; `min()`, `max()` and `sum()` support basic numerical analysis.

Iteration

- A loop can visit each list element to compute or transform values.

4. Relevant Course Outcome (CO-PO Mapping)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO5	2	2	3	2	3	-	1	1	1	1	2

Legend: 3 = High | 2 = Medium | 1 = Low | - = No Mapping

5. Lab Session Outcomes (LSOs)

- Create and update Python lists.
- Process numerical data using built-in functions and loops.
- Compute minimum, maximum and mean values.

6. Resources Required

Sr.	Resource	Specification
1	Computer	Standard laboratory computer
2	Python Environment	Python 3.x / IDLE / VS Code

7. Suggested Procedure

1. Create a list containing at least five numerical measurements.
2. Display the list and its length.
3. Append one new measurement.
4. Find minimum, maximum and sum using built-in functions.
5. Calculate mean as `sum(values)/len(values)`.
6. Iterate through the list and display each item with its position.
7. Optionally accept values from the user and build the list dynamically.

8. Activity / Implementation

- Example list: [12.5, 14.2, 13.8, 15.1, 11.9].
- Verify the calculated mean manually for one dataset.

9. Sample Output / Observations

Sr. No.	Step / Operation Performed	Output / Behaviour
1	List creation	Numerical values stored successfully
2	<code>append()</code>	New value added

Sr. No.	Step / Operation Performed	Output / Behaviour
3	min/max	Extreme values identified
4	Mean	Average computed correctly
5	Iteration	All elements processed

10. Result

Python lists were used to store, update, process and analyze numerical data successfully.

11. Interpretation of Results

- Lists simplify processing of sequential measurements.
- Built-in functions reduce code but results should still be interpreted and checked.
- The mean can be affected by extreme values.

12. Cautions

- Do not calculate the mean of an empty list.
- Ensure list elements are numeric before applying sum/min/max.
- Keep units consistent across all measurements.

13. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define a Python list.	✓		
2	State whether Python lists are mutable.	✓		
3	Name two built-in functions used to obtain minimum and maximum list values.	✓		
4	Explain the purpose of append().		✓	
5	Explain the value returned by len().		✓	
6	Explain the calculation of mean from a list of numerical values.		✓	
7	Write code to find the maximum value in a list.			✓
8	Add a new measurement value to an existing Python list.			✓
9	Calculate the mean of the list [10, 20, 30, 40].			✓

Legend :

R = Remember | U = Understand | A = Apply